EWB VPC PEERING PROJECT REPORT

AWS Cloud Networking Practical

Student	Bhavesh Dnyaneshwar Patil
Project	VPC Peering between two Amazon VPCs
AWS Region	Asia Pacific (Mumbai) — ap-south-1
Completion date	23 October 2026
Batch	To be filled in by student

Evidence in this report is taken from the screenshots provided. Screenshots are grouped under the matching AWS resource or connectivity test; missing evidence is explicitly noted rather than fabricated.

1. Objective

Connect two Amazon Virtual Private Clouds (VPCs) using VPC Peering and verify private communication between EC2 instances using private IP addresses. The two application instances have no public IPv4 address; access is demonstrated through the bastion host and private network paths.

2. Architecture and Network Design

VPC A uses CIDR 10.10.0.0/16 and contains a public subnet for the bastion host plus a private subnet for ewb-app-a. VPC B uses CIDR 10.20.0.0/16 and contains a private subnet for ewb-app-b. The VPC peering connection is named ewb-a-to-b-peering and is shown as Active in the console screenshot.

Internet	VPC A — 10.10.0.0/16	Peering	VPC B — 10.20.0.0/16
Internet Gateway	Public subnet 10.10.1.0/24 ewb-bastion 10.10.1.10 / public 13.206.218.134	pcx-009bcaafb4eede9d5 Status: Active	Private subnet 10.20.1.0/24 ewb-app-b 10.20.1.10 (private only)
	Private subnet 10.10.2.0/24 ewb-app-a 10.10.2.10 (private only)		

3. Resource Inventory

Resource	Name / ID	Address / Notes
VPC A	vpc-0920542ff0b68e64d	10.10.0.0/16
VPC B	ID not clearly shown in supplied screenshots	10.20.0.0/16
Public subnet	ewb-a-pulic / subnet-05390327285d4b1c1	10.10.1.0/24
Private subnet A	ewb-a-private / subnet-0fc39d4e37fb9e85a	10.10.2.0/24
Private subnet B	ewb-b-private / subnet-036f4caa014704bb3	10.20.1.0/24
Bastion EC2	ewb-bastion / i-0c2899eef34e918a0	10.10.1.10; public 13.206.218.134
Application EC2 A	ewb-app-a / i-0c56dfe7afab3ff08	10.10.2.10; no public IP
Application EC2 B	ewb-app-b / i-0de0a9bc85341a2f5	10.20.1.10; no public IP
VPC Peering	ewb-a-to-b-peering / pcx-009bcaafb4eede9d5	Active
Route tables	ewb-rt-a-public; ewb-rt-a-private; ewb-rt-b-private	IDs listed in evidence pages
Security groups	ewb-sg-bastion; ewb-sg-app-a; ewb-sg-app-b	Inbound rule screenshots included

Note: the public subnet name is written as “ewb-a-pulic” because that is how it appears in the AWS console screenshot. The VPC B ID is not confidently legible in the supplied evidence, so it is not guessed.

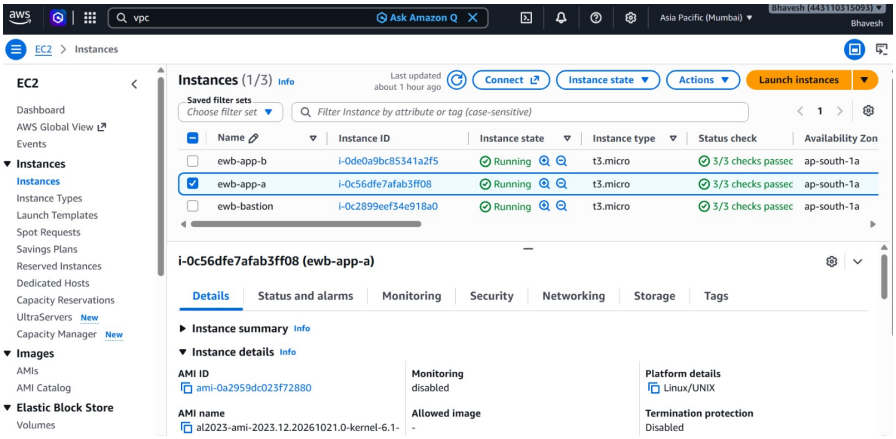
4. EC2 Instances

The EC2 instance-list screenshots are placed under the matching instance name. The first two screenshots below intentionally use the correct mapping: ewb-app-a and ewb-app-b, respectively.

Evidence 1 — ewb-app-a instance list and details

Console: EC2 console (Mumbai) **Check:** Select ewb-app-a

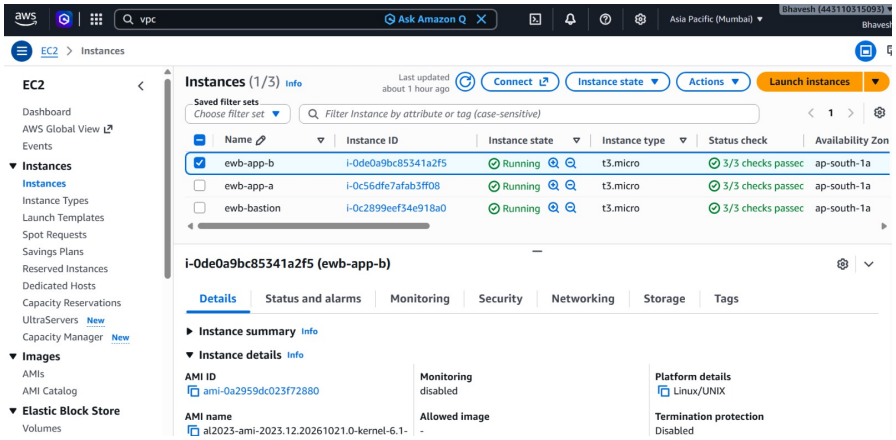
Observed result: ewb-app-a is Running and its status checks pass.



Evidence 2 — ewb-app-b instance list and details

Console: EC2 console (Mumbai) **Check:** Select ewb-app-b

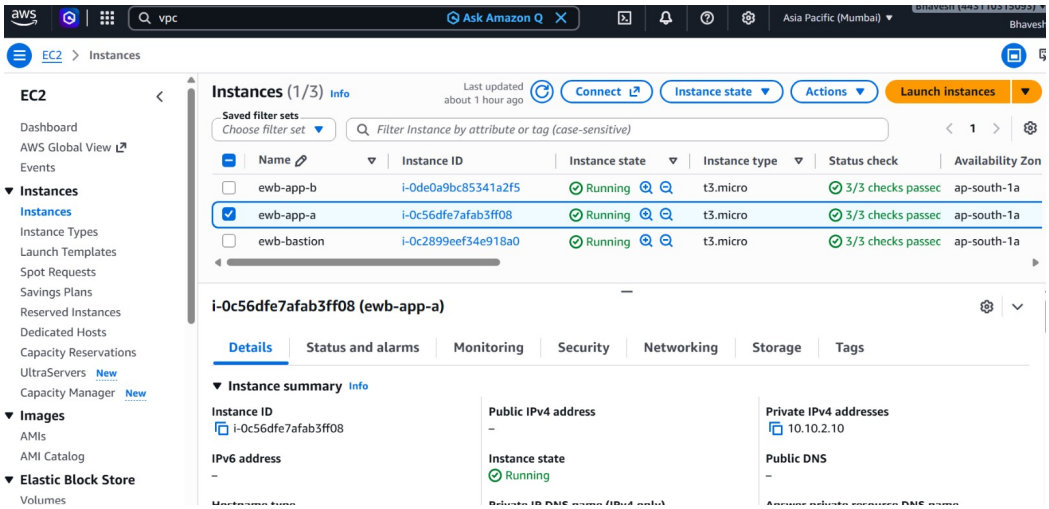
Observed result: ewb-app-b is Running and its status checks pass.



Evidence 3 — ewb-app-a instance summary

Console: EC2 console **Check:** Review network summary

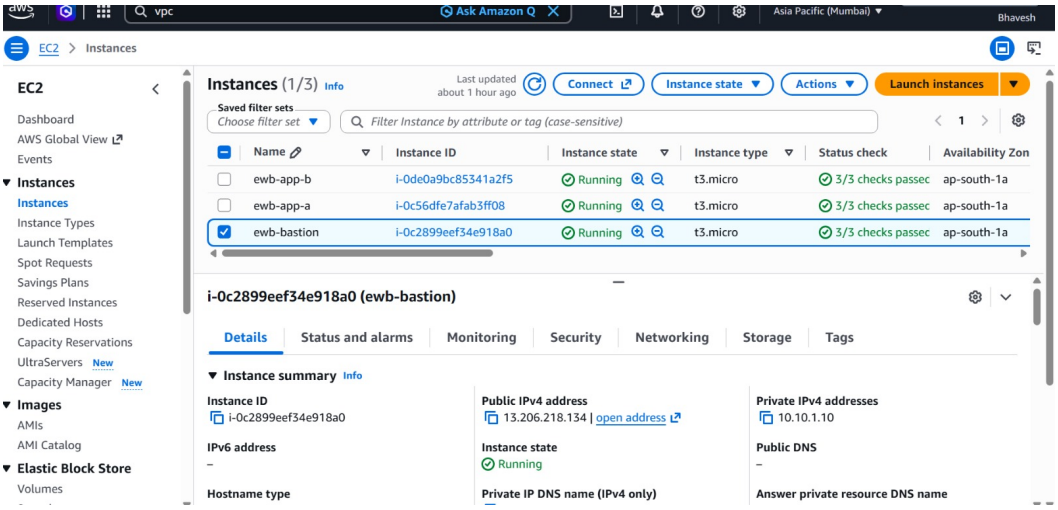
Observed result: Private IPv4 address 10.10.2.10; Public IPv4 address is blank.



Evidence 4 — ewb-bastion instance summary

Console: EC2 console **Check:** Review network summary

Observed result: Private IPv4 address 10.10.1.10 and public IPv4 address 13.206.218.134.

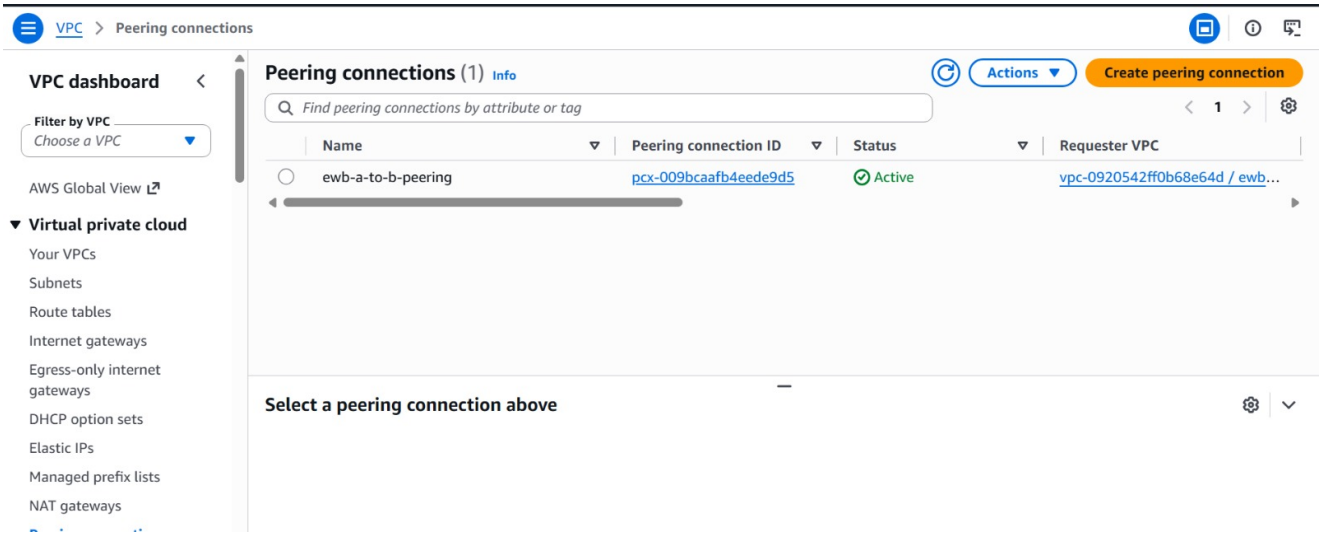


5. VPC Peering Connection

Evidence 5 — Active VPC peering connection

Console: VPC console → Peering connections **Check:** Check connection status

Observed result: ewb-a-to-b-peering (pcx-009bcaafb4eede9d5) is shown as Active.



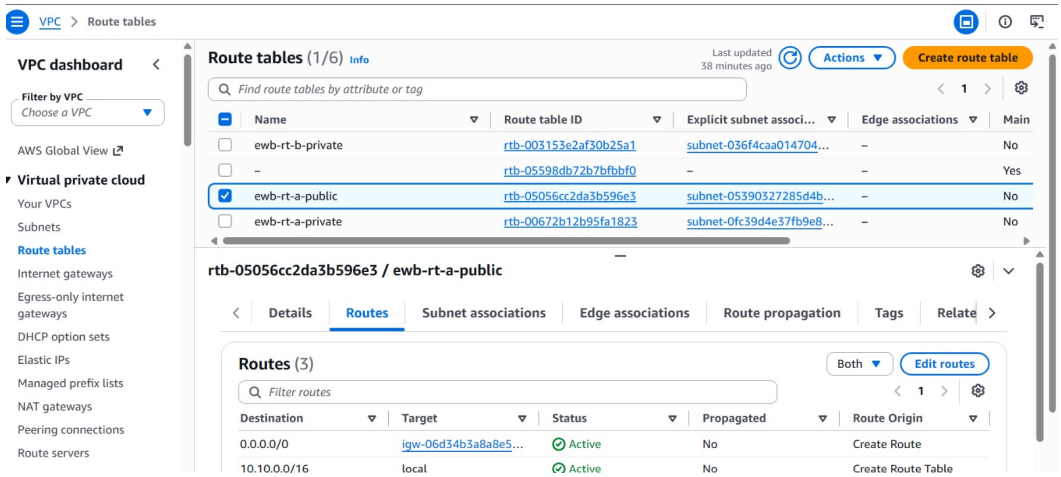
The supplied screenshot confirms the peering connection status. A separate peering-details screenshot showing both requester/accepter VPC IDs and CIDRs was not included.

6. Route Tables and Subnet Associations

Evidence 6 — ewb-rt-a-public routes

Console: VPC console → Route tables **Check:** Open Routes tab for ewb-rt-a-public

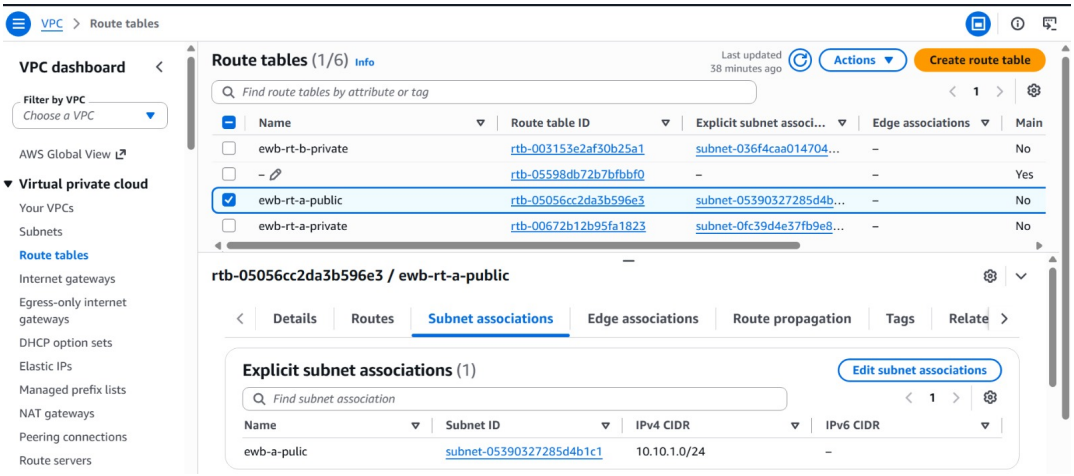
Observed result: The default route 0.0.0.0/0 points to an Internet Gateway; the local VPC route is also visible.



Evidence 7 — ewb-rt-a-public subnet association

Console: VPC console → Route tables **Check:** Open Subnet associations tab

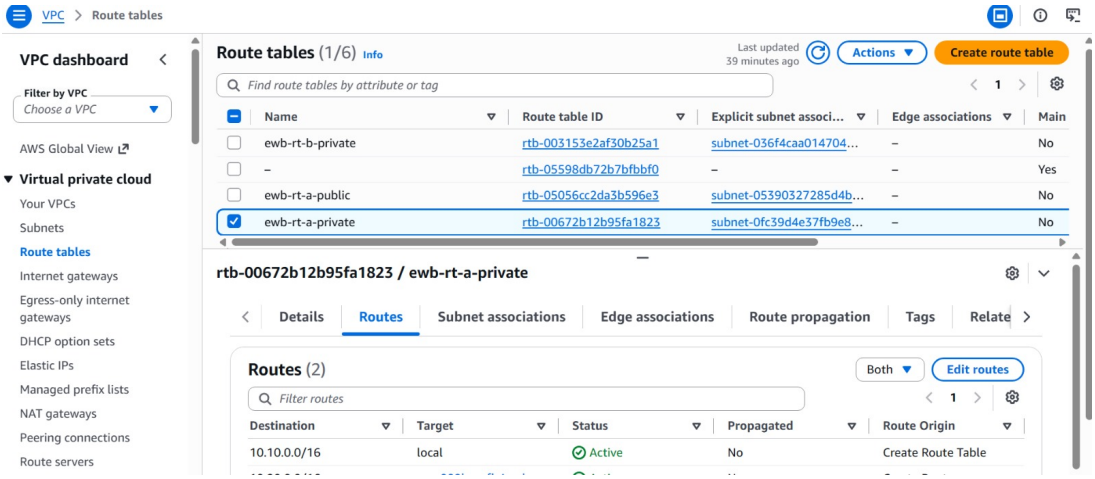
Observed result: Associated with the public subnet ewb-a-pulic, CIDR 10.10.1.0/24.



Evidence 8 — ewb-rt-a-private routes

Console: VPC console → Route tables **Check:** Open Routes tab for ewb-rt-a-private

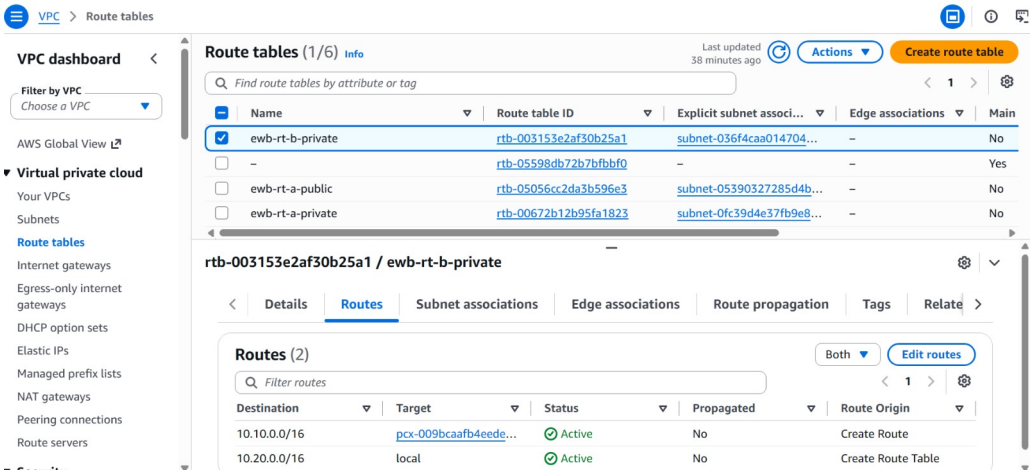
Observed result: The screenshot shows the local route, but the peering route row is cut off and cannot be verified from this image.



Evidence 9 — ewb-rt-a-private subnet association

Console: VPC console → Route tables **Check:** Open Subnet associations tab

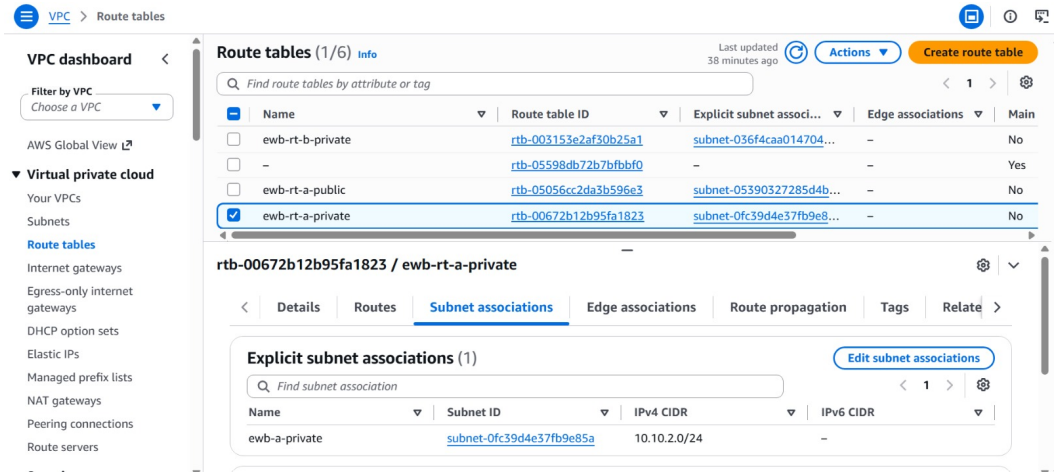
Observed result: Associated with ewb-a-private, CIDR 10.10.2.0/24.



Evidence 10 — ewb-rt-b-private routes

Console: VPC console → Route tables **Check:** Open Routes tab for ewb-rt-b-private

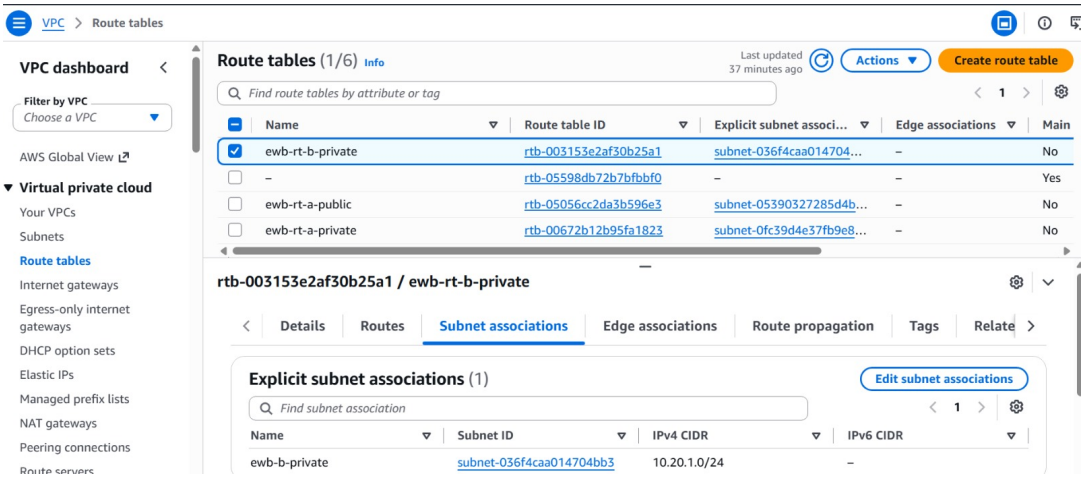
Observed result: The screenshot shows 10.10.0.0/16 via the peering connection and 10.20.0.0/16 as local; both are marked Active.



Evidence 11 — ewb-rt-b-private subnet association

Console: VPC console → Route tables **Check:** Open Subnet associations tab

Observed result: Associated with ewb-b-private, CIDR 10.20.1.0/24.

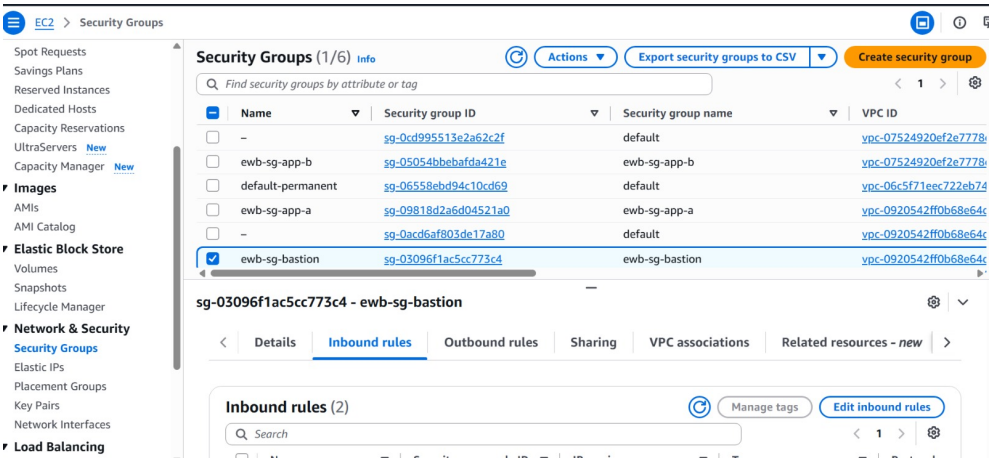


7. Security Groups — Inbound Rules

Evidence 12 — ewb-sg-bastion selected

Console: EC2 console → Security Groups **Check:** Select ewb-sg-bastion

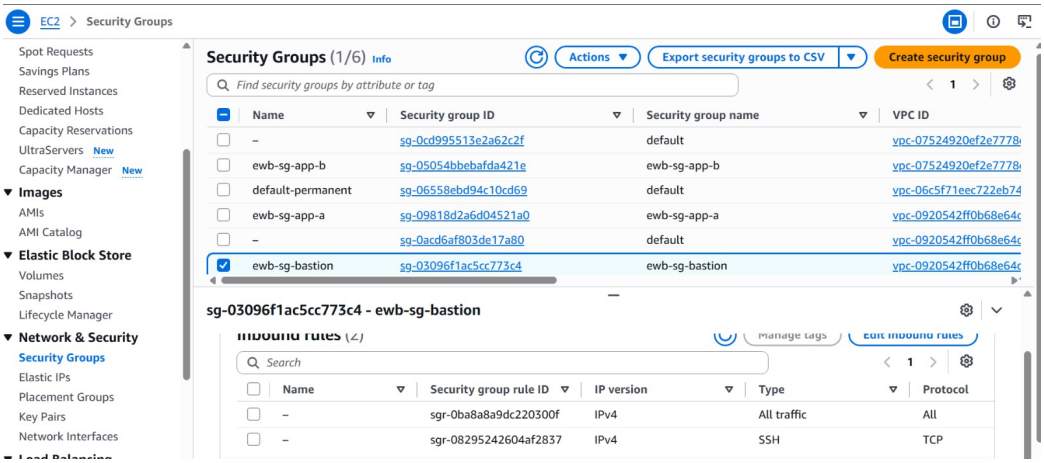
Observed result: The security group is selected and its inbound rules section is visible.



Evidence 13 — ewb-sg-bastion inbound rules

Console: EC2 console → Security Groups **Check:** Review inbound rule list

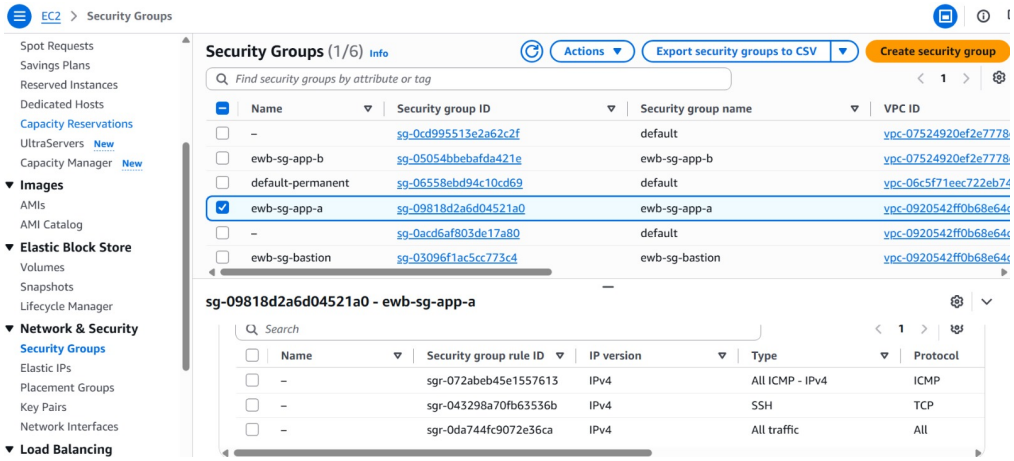
Observed result: The visible rule types include All traffic and SSH (TCP). Source/port details are not fully visible.



Evidence 14 — ewb-sg-app-a inbound rules

Console: EC2 console → Security Groups **Check:** Select ewb-sg-app-a

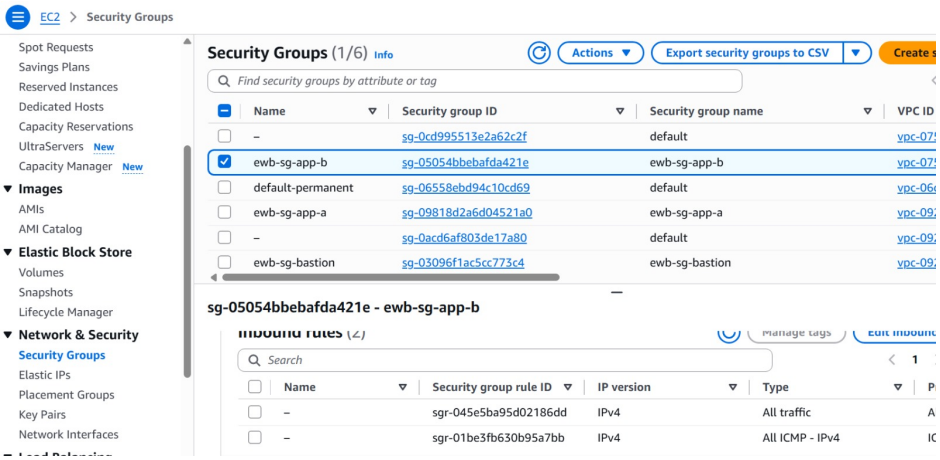
Observed result: Visible rule types include All ICMP - IPv4, SSH (TCP), and All traffic.



Evidence 15 — ewb-sg-app-b inbound rules

Console: EC2 console → Security Groups **Check:** Select ewb-sg-app-b

Observed result: Visible rule types include All traffic and All ICMP - IPv4. Source/port details are not fully visible.

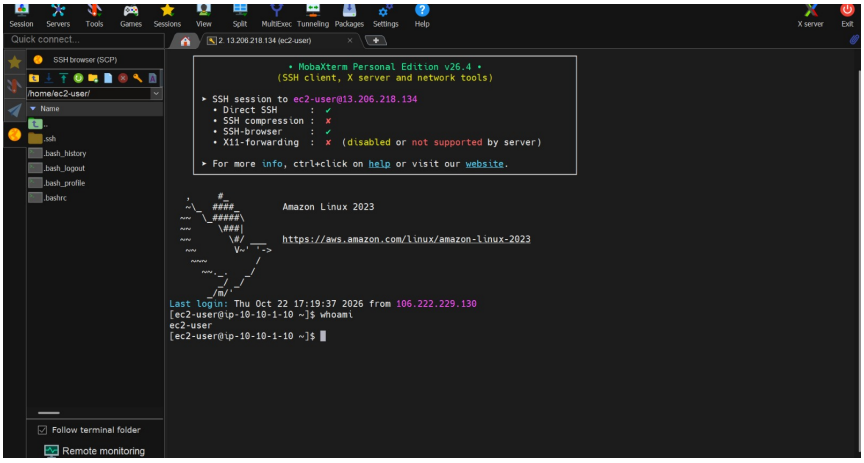


8. Connectivity Tests

Evidence 16 — SSH login to bastion host

Console: MobaXterm **Check:** Connect to 13.206.218.134 and run whoami

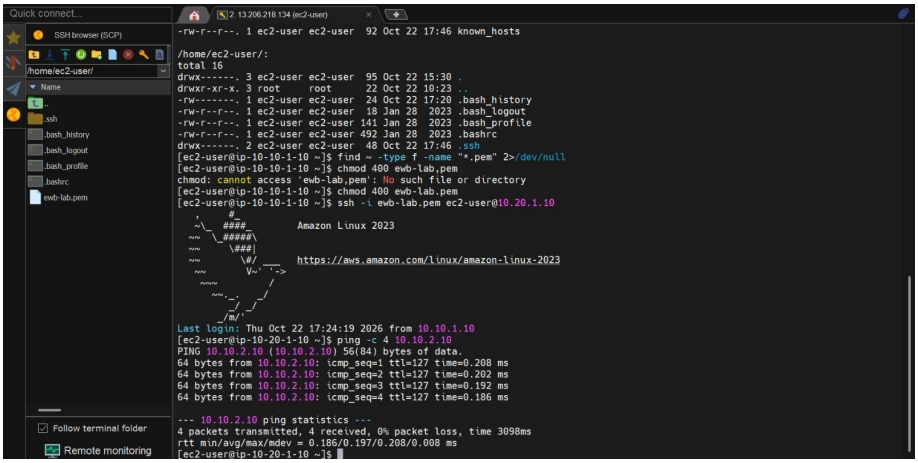
Observed result: SSH login succeeds; the shell identifies the user as ec2-user on the bastion host.



Evidence 17 — SSH from bastion to ewb-app-b and ping back to VPC A

Console: MobaXterm / SSH terminal **Check:** Connect to 10.20.1.10 and ping 10.10.2.10

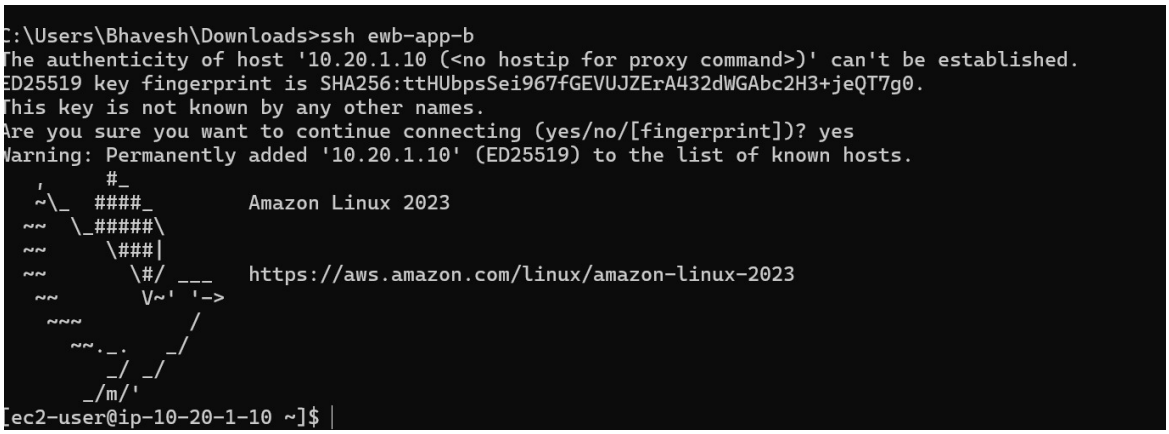
Observed result: The screenshot shows four replies from 10.10.2.10 and 0% packet loss.



Evidence 18 — SSH connection to ewb-app-b

Console: Windows terminal **Check:** Run ssh ewb-app-b

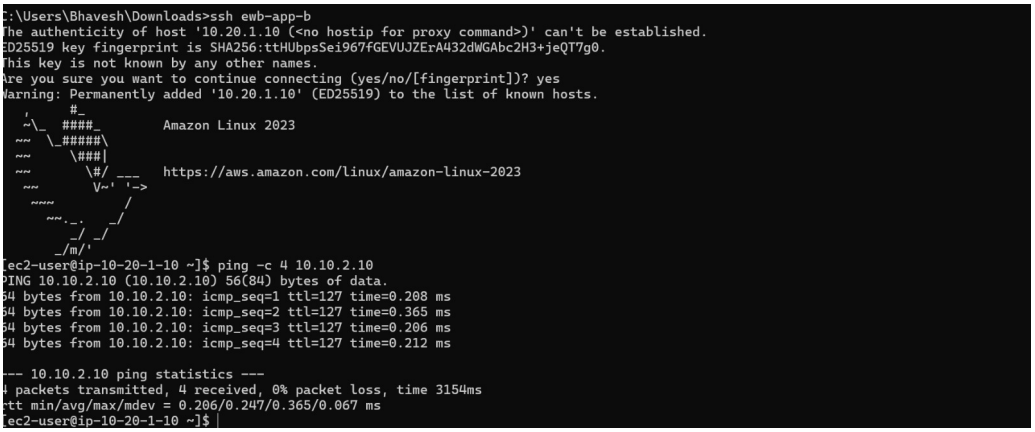
Observed result: The host-key prompt is accepted and the Amazon Linux 2023 shell opens for 10.20.1.10.



Evidence 19 — Reverse ping from ewb-app-b to ewb-app-a

Console: SSH terminal on ewb-app-b **Check:** Run ping -c 4 10.10.2.10

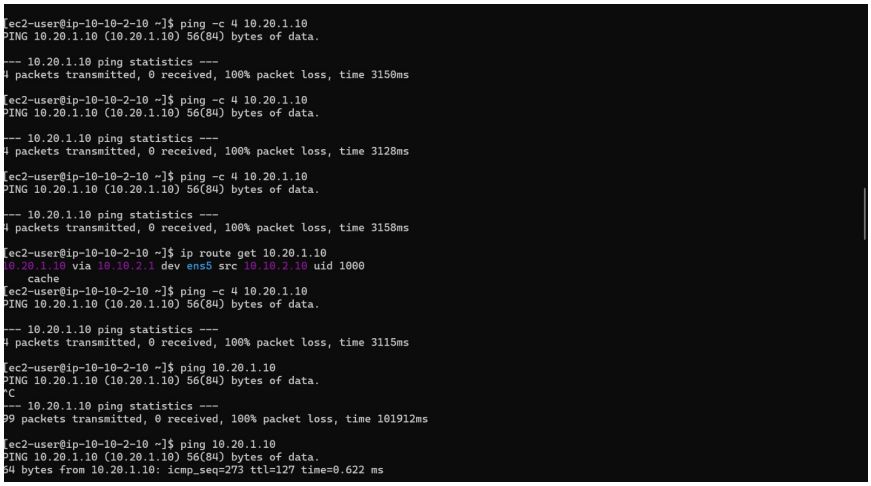
Observed result: Four packets are transmitted and four received, with 0% packet loss.



Evidence 20 — Initial ping failures from ewb-app-a to ewb-app-b

Console: SSH terminal on ewb-app-a **Check:** Ping 10.20.1.10 and inspect route

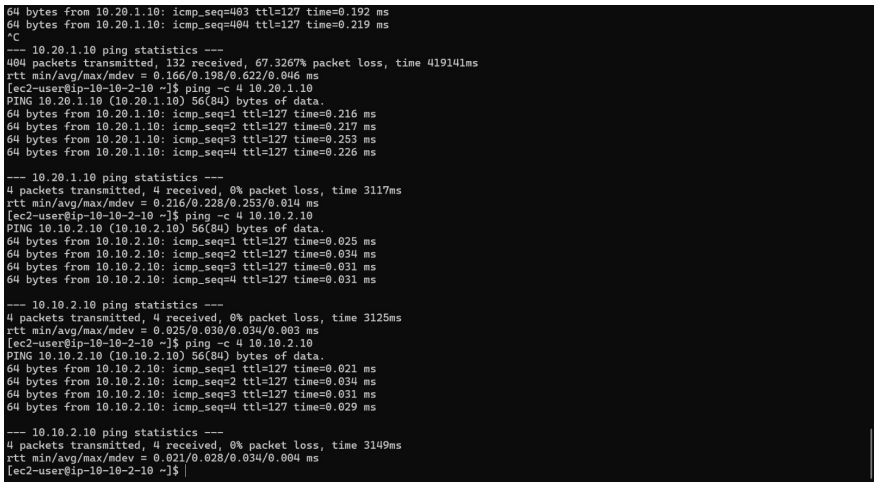
Observed result: The screenshot records initial 100% packet loss and an ip route get result via 10.10.2.1. It does not establish the exact cause of the initial failure.



Evidence 21 — Successful ping from ewb-app-a to ewb-app-b

Console: SSH terminal on ewb-app-a **Check:** Run ping -c 4 10.20.1.10

Observed result: A four-packet test succeeds with 4 received and 0% packet loss. The screenshot also contains local ping tests to 10.10.2.10; those are not VPC-peering tests.



9. Forward and Return Traffic Path

Forward path (VPC A to VPC B): ewb-app-a (10.10.2.10) should use ewb-rt-a-private to route 10.20.0.0/16 through the peering connection pcx-009bcaafb4eede9d5. The screenshot for this route table does not show the full peering-route row, so that route is described based on the project configuration and successful connectivity test, but is not fully visible in the supplied route-table evidence.

Return path (VPC B to VPC A): ewb-app-b (10.20.1.10) uses ewb-rt-b-private, whose screenshot shows 10.10.0.0/16 routed to the peering connection. The successful reverse ping supports private connectivity in this direction.

10. Cleanup

The user reported that the EC2 instances were terminated and the lab resources were cleaned up. A cleanup screenshot was not supplied, so the cleanup is recorded as user-reported and is not presented as screenshot-verified evidence.

11. What I Learned

This practical covered creating two VPCs with non-overlapping CIDR ranges, configuring subnets and route tables, creating a VPC peering connection, reviewing security group inbound rules, connecting to private instances through a bastion host, and validating private connectivity with SSH and ping.

Evidence gaps / improvements

- Add a clear ewb-app-b instance summary showing private IP 10.20.1.10 and no public IP.
- Add a full ewb-rt-a-private Routes screenshot where the 10.20.0.0/16 peering route and target are readable.
- If required by the evaluator, add wider security-group screenshots showing the Source and Port range columns.
- Add cleanup evidence only if a screenshot or console record is available.

Report prepared from the screenshots supplied for this project. No missing screenshots or unverified AWS details have been fabricated.